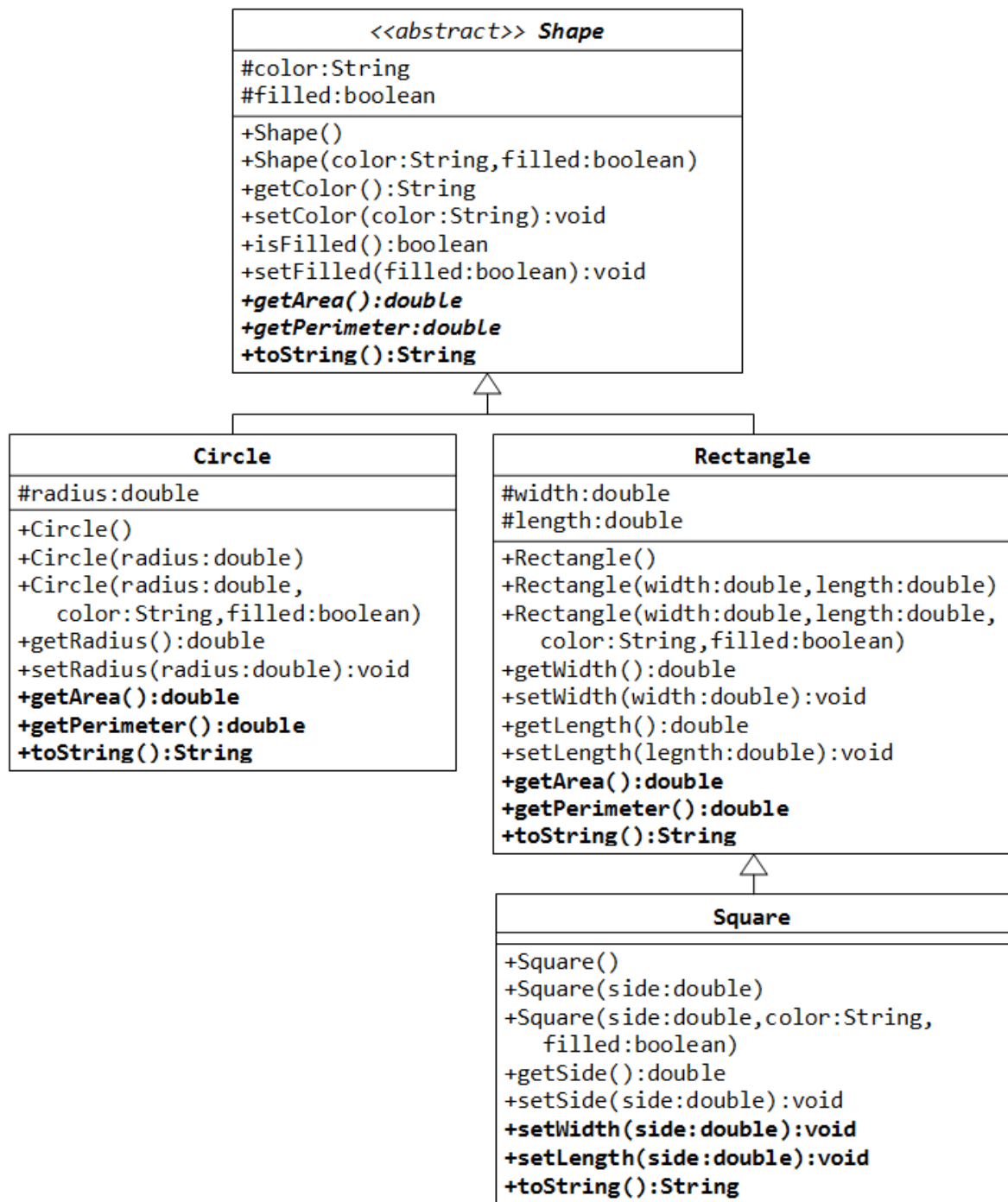


Tutorial 2: Advanced OOP

Req1: Create a “Shape” abstract class, which contains the following attributes and methods



- Two protected instance variables `color(String)` and `filled(boolean)`. The protected variables can be accessed by its subclasses and classes in the same package. They are denoted with a '#' sign in the class diagram.
- Getter and setter for all the instance variables, and `toString()`.
- Two abstract methods `getArea()` and `getPerimeter()` (shown in italics in the class diagram).

Write main method to initiate 3 objects, 1 for circle, 1 for Rectangle and 1 for Square, initiate with corresponding constructor and use `toString()` method to print out perimeter and area of each shape.

Req 2:

Provide an interface `Measurable` with a method `double getMeasure()` that measures an object in some way. Make `Employee` implement `Measurable`. The method `getMeasure()` will return salary of employee object.

Attribute of `Employee` includes:

- + `String name;`
- + `double salary;`

Create a class `MeasureCalculation` which provides a method `double average(Measurable[] objects)` that computes the average measure. Use it to compute the average salary of an array of employees.

Example output

```
public class MyClass {  
  
    public static void main(String[] args) {  
        Measurable[] m = new Measurable[3];  
        m[0] = new Employee("camnh", 0);  
        m[1] = new Employee("tranglt", 10000);  
        m[2] = new Employee("congnv", 2000);  
  
        System.out.println("Average is " + MeasureCalculation.average(m));  
    }  
}
```



```
Average is 4000.0
```

Req 3

Continue with the preceding exercise and provide a method `Measurable largest (Measurable[] objects)` to class `MeasureCalculation` to find highest value. Use it to find employee name with highest salary among employees. Why do you need a cast?

Example output

Result

CPU Time: 0.08 sec(s), Memory: 23520 kilobyte(s)

```
Average is 4000.0
Max is tranglt
```

Note:

1. You can upload your custom libraries to /lib folder
2. For file operations, read and write to '/myfiles' folder
3. For detailed documentation check - [Our Documentation](#)

Req 4

Create new class call `BankAccount` with the following attribute

- + String `accountName`;
- + double `balanceNumber`;

In order to integrate with existing codebase structure with minimal modification, what is the pre-condition for creating this class. If this class exist, make changes in needed parts to integrate with existing codebase with minimal modifications.

Answer

Req 2

```
public interface Measurable
{
    public double getMeasure();
}
```

```
public class Employee implements Measurable
{
    private double salary;
    private String name;

    public Employee (String name, double salary)
    {
        this.name = name;
        this.salary = salary;
    }

    public String getName()
    {
        return this.name;
    }

    public double getMeasure()
    {
        return this.salary;
    }
}
```

```
public class MeasureCalculation
{
    public static double average(Measurable[] objects)
    {
        double sum = 0;
        for(Measurable obj : objects)
        {
            sum += obj.getMeasure();
        }
        return sum/(objects.length);
    }
}
```

```

public class MyClass {

    public static void main(String[] args) {
        Measurable[] m = new Measurable[3];
        m[0] = new Employee("camnh", 0);
        m[1] = new Employee("tranglt", 10000);
        m[2] = new Employee("congnv", 2000);

        System.out.println("Average is " + MeasureCalculation.average(m));
    }
}

```

Req 3

```

public static Measurable largest(Measurable[] objects)
{
    double max = 0;
    Measurable temp = null;
    for(Measurable obj : objects)
    {
        if (obj.getMeasure() > max) {
            max = obj.getMeasure();
            temp = obj;
        }
    }

    return temp;
}

public class MyClass {

    public static void main(String[] args) {
        Measurable[] m = new Measurable[3];
        m[0] = new Employee("camnh", 0);
        m[1] = new Employee("tranglt", 10000);
        m[2] = new Employee("congnv", 2000);

        System.out.println("Average is " + MeasureCalculation.average(m));
        System.out.println("Max is " + ((Employee)MeasureCalculation.largest(m)).getName());
    }
}

```

The reason why we need a cast is because return type of method `largest()` is an interface, which doesn't cast to specific class, so we need a cast to convert from Interface -> class

Req4:

The BankAccount class needs to implement Measurable interface

Create BankAccount

```
public class BankAccount implements Measurable
{
    private double balanceNumber;
    private String accountName;

    public Employee (String accountName, double balanceNumber)
    {
        this.accountName = accountName;
        this.balanceNumber = balanceNumber;
    }

    public String getName()
    {
        return this.accountName;
    }

    public double getMeasure()
    {
        return this.balanceNumber;
    }
}
```

Only change class Main, no need to change others

```
public class MyClass {

    public static void main(String[] args)
    {
        Measurable[] m = new Measurable[3];
        m[0] = new BankAccount("camnh", 0);
        m[1] = new BankAccount("tranglt", 4000);
        m[2] = new BankAccount("congnv", 6000);

        System.out.println("Average is " + MeasureCalculation.average(m));
        System.out.println("Max is " + ((BankAccount)MeasureCalculation.largest(m)).getName());

    }
}
```